

Another structure that you may find useful while writing code is the bulk transfer:

```
struct usbdevfs_bulktransfer {
    unsigned int ep;
    unsigned int len;
    unsigned int timeout; /* in milliseconds */
    void __user *data;
};
```

You may also find the 'names given to the constants' useful, while writing the drivers:

```
#define USBDEVFS_URB_SHORT_NOT_OK 0x01
#define USBDEVFS_URB_ISO_ASAP      0x02
#define USBDEVFS_URB_NO_FSBR       0x20
#define USBDEVFS_URB_ZERO_PACKET 0x40
#define USBDEVFS_URB_NO_INTERRUPT 0x80

#define USBDEVFS_URB_TYPE_ISO      0
#define USBDEVFS_URB_TYPE_INTERRUPT 1
#define USBDEVFS_URB_TYPE_CONTROL 2
#define USBDEVFS_URB_TYPE_BULK     3
```

How hard is it to write a USB driver?

Since you have all these tools (especially, the *ioctl*s, and information about structures), you can start writing a USB device driver, provided you are a power user. I know that beginners may find it hard to start off at this stage with just the information available in this article. So I suggest you get a copy of the USB specifications—you can ask Google to fetch the URL for you—so that you know where to start.

What about the programmers of intermediate level? Well, you may face various hurdles. But you don't have to worry; I'm going to introduce you to a new library that resides on top of these tools and provides a simple interface—*libusb*!

You can get a copy of the library from <http://www.libusb.org/>. Then you can use the following functions to perform all the basic tasks for you:

```
usb_init();
usb_find_busses();
usb_find_devices();
```

If you want to exit, you can just use the API provided by this and it will take care of all the housekeeping work for you:

```
API_EXPORTED void libusb_exit(struct libusb_context *ctx)
{
    USBI_GET_CONTEXT(ctx);
    usb_dbg("");

    /* a little sanity check. doesn't bother with open_devs locking because
```

```
    * unless there is an application bug, nobody will be accessing this.
}

if (!list_empty(&ctx->open_devs))
    usb_warn(ctx, "application left some devices open");

usb_iio_exit(ctx);
if (usb_backend->exit)
    usb_backend->exit();

pthread_mutex_lock(&default_context_lock);
if (ctx == usb_default_context) {
    usb_dbg("freeing default context");
    usb_default_context = NULL;
}
pthread_mutex_unlock(&default_context_lock);

free(ctx);
}
```

Writing code using *libusb*

Let us assume that we want to print the names of all the USB devices attached to your system. Well, you can do that without using *libusb*. But if you use the library, the code will appear more compact.

Let me write a simple code and demonstrate how to use *libusb* to do this:

```
#include <stdio.h>
#include <sys/types.h>
#include <libusb/libusb.h>

static void print_all_devices(libusb_device **devs)
{
    //write all declarations here

    while ((mydevice = devs[i++]) != NULL) {
        struct libusb_device_descriptor desc;
        int retval = libusb_get_device_descriptor(mydevice, &desc);

        //perform check for retval

        printf("%04x:%04x (bus %d, device %d)\n",
            desc.idVendor, desc.idProduct,
            libusb_get_bus_number(mydevice), libusb_get_device_
            address(mydevice));
    }
}

int main(void)
{
    libusb_init(NULL);
    cnt = libusb_get_device_list(NULL, &devs);

    //write code to the necessary checks for cnt
```